Mechanized Soundness of a Byte-Level Linked AIR/FRI Verifier in the Finite Classical ROM: From Lean 4 Proofs to a Hardened Rust Implementation

Juan Diego Susunaga Velasquez
School of Engineering, Science and Technology
Universidad del Rosario
Bogotá, Colombia
juand.susunaga@urosario.edu.co

Pedro Mario Wightman Rojas
School of Engineering, Science and Technology
Universidad del Rosario
Bogotá, Colombia
pedro.wightman@urosario.edu.co

Abstract*—Interactive Oracle Proofs of Proximity (IOPPs) such as FRI and Algebraic Intermediate Representations (AIR) are typically analyzed in idealized algebraic models where commitments, Fiat–Shamir challenges, and query selections act as independent steps. In contrast, deployed verifiers parse raw byte streams, share stateful hash transcripts across phases, execute bounded deduplication loops, and check proof-of-work (grinding) nonces. We present an axiom-clean mechanization in Lean 4 (2,709 verified declarations across 347 submodules) of the soundness of a byte-level linked AIR/FRI verifier in the finite classical Random Oracle Model (ROM) over the Goldilocks prime field ($p = 2^{64} - 2^{32} + 1$). Our formalization models the canonical byte parser, a stateful ROM table under Q adaptive queries, causal oracle freezing, and a bounded selector extracting 40 distinct query pairs within $R = 512$ draws. We formally refute naive "universal freshness" across multi-root chains, bound the selector coupling error $\delta(B)$, and compose $g \in \{12, 20\}$ grinding bits over at most $Q + 1$ eligible state/nonce pairs (e.g., $\Pr[\mathsf{AcceptFalse}_{12}] < 0.004757335454$ for $T = 4096$, $g = 20$, $Q = 2^{20}$). Finally, we describe how the formalization guided the hardening of our Rust verifier, eliminating legacy acceptance flaws at relative Hamming distance 7/8.

***Index Terms*—Interactive Oracle Proofs, FRI, STARK, Random Oracle Model, Lean 4, Formal Verification, Rust.**

I. Introduction

Succinct arguments based on Algebraic Intermediate Representations (AIR) and Fast Reed–Solomon Interactive Oracle Proofs of Proximity (FRI) [4], [5] underpin modern verifiable computation systems and Layer-2 (L2) validity rollups.

While the theoretical soundness of FRI and DEEP-ALI has been extensively refined in the cryptographic literature [6], [12], a significant gap persists between idealized paper proofs and deployed byte-level verifiers:

1) **Byte-Level Parsing and Pre-Hash Validation:** Paper analyses assume well-typed algebraic objects. Real verifiers parse untrusted byte slices where non-canonical field elements ($x \geq p$), trailing bytes, or attacker-controlled vector capacities can cause memory exhaustion or transcript malleability prior to any hash invocation.
2) **Shared Stateful ROM vs. Idealized Freshness:** Fiat–Shamir transformations [7], [10] are often analyzed by treating each squeeze as an independent uniform draw. In a concrete transcript absorbing sequential Merkle roots and nonces over a single hash function, an adaptive adversary can pre-query, repeat, or interleave oracle calls. Assuming "universal freshness" across chained root absorptions is mathematically impossible once two or more roots are absorbed.
3) **Bounded Deduplication and Shared Queries:** To sample $q = 40$ distinct query pairs from $M = 2^{k+2}$ folded pairs without replacement, concrete verifiers squeeze indices iteratively and discard duplicates up to a fixed cutoff ($R = 512$ draws). Moreover, the same 40-pair sample is shared simultaneously across the AIR check and two linked FRI executions (trace and composition), invalidating naive product bounds.
4) **Adaptive Grinding Composition:** Proof-of-work (grinding) of g bits is frequently accounted for by multiplying the soundness error by $2^{-g}Q$. Justifying this factor requires proving that every accepting execution maps to a first-exposed state/nonce pair in the ROM history, that at most $Q + 1$ such pairs exist across all retries, and that the 2^{-g} mass composes locally with the coupled selector before query responses are observed.

In this paper, developed within the **NEXUS** research project, we bridge this gap by mechanizing the complete byte-level linked AIR/FRI verifier in **Lean 4** (v4.35.0 with `mathlib4`) and aligning our **Rust** implementation to the resulting formal contract.

A. Contributions

- **End-to-End Byte-Level ROM Formalization in Lean 4:** We formalize the canonical byte reader,

shape validator, Fiat–Shamir channel program, six-position AIR aperture verifier, and dual linked FRI verifier over the Goldilocks field $\mathbb{F}_p$. The mechanization totals **2,709 declarations across 347 submodules**, audited with zero `sorryAx`.

- **Memory-Aware Coupled Selector and Causal Freezing:** We prove that an adaptive adversary (Q queries) interacting with a stateful ROM coincides with lazy uniform sampling over finite effective prefixes. We establish causal oracle freezing prior to first exposure and bound the total variation distance between the $R = 512$ selector and an ideal hypergeometric draw via $\delta(B)$.
- **Machine-Checked Concrete Soundness Bounds:** We derive closed-form rational bounds for fixed profiles ($k \in \{4, \ldots, 12\}$, $T = 2^k$ rows) and adaptively chosen profiles, charging hash-log collisions (Bad) and algebraic AIR/FRI exceptions once globally while weighting the selector risk by 2^{-g} across at most $Q+1$ pairs.
- **Co-Design and Hardening of the Rust Verifier:** We document how the formalization guided the redesign of `nexus-zk::stark::linked` and `nexus-active::linked_verifier`, eliminating legacy soundness flaws (demonstrated by three counterexample traces at relative distance 7/8 that passed the legacy verifier) and enforcing external context binding.
- **Explicit Epistemic Boundaries:** We rigorously separate what the Lean 4 theorems prove in the finite classical ROM from what remains outside the mechanized model (concrete SHA3-256 cryptanalysis, universal machine-code refinement, and full Quantum Random Oracle Model [QROM] transfer).

II. Protocol Specification and Threat Model

A. Algebraic Domain and Linked AIR/FRI Structure

The verifier operates over the Goldilocks prime field $\mathbb{F}_p$ where $p = 2^{64} - 2^{32} + 1 = 18{,}446{,}744{,}069{,}414{,}584{,}321$, and its quadratic extension $\mathbb{F}_{p^2} = \mathbb{F}_p[u]/(u^2 - 7)$ for folding challenges. An execution profile is parameterized by $k \in \{4, 5, \ldots, 12\}$, corresponding to a trace length of $T = 2^k \in [16, 4096]$ rows.

With a blowup factor of 8, the Low-Degree Extension (LDE) evaluation domain has size $N = 8T = 2^{k+3}$ over a multiplicative coset $D = 7 \cdot \langle \omega_N \rangle \subset \mathbb{F}_p^\times$, where $\omega_N^8 = g_T$ generates the trace subgroup of order T and $\omega_N^{N/2} = -1$.

We verified exhaustively across all 65,408 coset points of the nine supported profiles that no vanishing denominators (poles at $x \in \{0, 1, g_T^{T-1}\}$ or $x^T = 1$) occur on D. Opposite points $(x, -x)$ are grouped into $M = N/2 = 2^{k+2}$ folded pairs of tree depth $d = k + 3$.

Unlike standalone FRI benchmarks, our verifier checks a **linked dual-FRI** relation:

1) **Trace Commitment & AIR Apertures:** For each sampled pair index $i \in \{0, \ldots, M-1\}$, the verifier authenticates six positions in $[0, N)$ against the shared trace Merkle root (the current and shifted trace rows on both x and $-x$, verified across 196,224 total index configurations).
2) **Composition Polynomial Link:** Using Fiat–Shamir challenges drawn after the trace root, the verifier evaluates the algebraic transition and boundary constraints at x and $-x$ and verifies that they match the initial layer of the composition commitment.
3) **Dual FRI Folding:** The verifier simultaneously executes $m_{\text{trace}} = k$ folding rounds for the trace polynomial and $m_{\text{comp}} = k + 1$ folding rounds for the composition polynomial over $\mathbb{F}_{p^2}$, using the *exact same* set of 40 sampled pair indices.

B. Byte-Level Certificate Grammar and Cost Budget

A serialized proof certificate (S, π) is a raw byte sequence (`List Octeto` in Lean, `&[u8]` in Rust) bounded by a hard cap of 2 MiB (2,097,152 bytes). For profile k ($d = k + 3$), the commitment byte size is

$$\mathsf{Bytes}_{\text{com}}(m) = 24 + 32m, \tag{1}$$

the per-phase FRI folding path size is

$$\mathsf{Bytes}_{\text{fri}}(m) = 8 + \sum_{j=0}^{m-1} \big(48 + 64(d - j)\big), \tag{2}$$

the per-query aperture size is

$$\begin{aligned}\mathsf{Bytes}_{\text{query}}(k) = {} & 8 + 6(16 + 32d) \\ & + \mathsf{Bytes}_{\text{fri}}(k) + \mathsf{Bytes}_{\text{fri}}(k+1),\end{aligned} \tag{3}$$

and the total certificate byte length is

$$\begin{aligned}\mathsf{Bytes}(k) = {} & 86 + \mathsf{Bytes}_{\text{com}}(k) + \mathsf{Bytes}_{\text{com}}(k+1) \\ & + 40 \cdot \mathsf{Bytes}_{\text{query}}(k).\end{aligned} \tag{4}$$

As shown in (1)–(4), the exact size of a valid certificate is completely determined by k. Table I summarizes the exact certificate sizes from (4) and the worst-case verifier hash budgets across all nine supported profiles.

Even for the largest profile ($k = 12$, $T = 4096$), a valid certificate occupies 760,286 bytes, well below the 2 MiB pre-allocation limit, and the verifier performs at most $V_{\max}(12) = 24{,}895$ random oracle calls (including up to $R = 512$ selector draws), even upon rejection.

C. Finite Classical ROM and Adaptive Adversary Model

We model the hash function as a single, shared classical Random Oracle $H : \mathcal{U} \to \{0,1\}^{256}$ (`Digest`) with persistent memory across the entire experiment:

- Each distinct query input $x \in \mathcal{U}$ returns an independent uniform 256-bit digest upon its *first* invocation; any subsequent query on the same x by either the adversary or the verifier deterministically returns the stored digest.

TABLE I
EXACT CERTIFICATE SIZES AND VERIFIER HASH CALL BUDGETS ($R = 512$)

Prof. k	**Rows** T	**Dom.** N	**Cert. Bytes**	**Max Hashes** $V_{\max}$
4	16	128	196,574	6,943
5	32	256	249,118	8,627
6	64	512	306,782	10,471
7	128	1,024	369,566	12,475
8	256	2,048	437,470	14,639
9	512	4,096	510,494	16,963
10	1,024	8,192	588,638	19,447
11	2,048	16,384	671,902	22,091
12	4,096	32,768	760,286	24,895

- The adaptive adversary (proponer, atacar, actualizar) is parameterized by an arbitrary state type A and may issue up to Q oracle queries across certificate preparation, grinding searches, and arbitrary interleavings.
- Together with the verifier's worst-case cost of 24,895 calls for $R = 512$, the total number of distinct oracle entries recorded during the joint execution is bounded by the shared budget $B = Q+24{,}895$, with an auxiliary log capacity parameter L.
- Grinding requires $g \in \{12, 20\}$ leading zero bits on the work digest absorbed with a 64-bit nonce prior to query index extraction.

III. MECHANIZED SOUNDNESS IN LEAN 4

Our Lean 4 formalization is organized into 347 submodules under the `Nexus` namespace, culminating in `Nexus.Verificador.CotaTrabajoFinal` and `Nexus.Verificador.CotaTrabajoNumerica`. Rather than axiomatizing idealized intermediate states, we prove soundness directly from the byte-level verifier program.

A. Refuting Naive Universal Freshness and Causal Oracle Freezing

A subtle pitfall in formalizing Fiat–Shamir transcripts for multi-round protocols (such as FRI, which absorbs $m \leq 30$ sequential commitment roots) is assuming that every absorbed state triggers a fresh oracle query with probability 1. In Lean 4, we proved that such a *universal freshness* hypothesis is mathematically contradictory as soon as two roots are absorbed, because an adaptive adversary can deliberately replay a previously seen root or state prefix.

Instead, we construct a memory-aware coupling over the finite ROM execution tree (`LeyArbol` and `ObservacionesPrefijo`):

1) **Hash Log Integrity (SinBad):** We define a computable detector over the execution log that flags any collision, pre-image inversion, or prefix cycle among recorded table entries. In `CotaSinBad`, we prove that the probability of violating SinBad under budget $B = Q + 24{,}895$ and auxiliary bound L satisfies

$$\Pr[\neg\mathsf{SinBad}] \leq \min\left\{1, \frac{B(49B + L)}{2^{256}}\right\}. \tag{5}$$

2) **Causal Freezing of Commitments (`PrefijoAIR`, `RondasCausales`):** Under SinBad in (5), before the first challenge response (E) of each FRI layer or AIR linear combination is drawn, the corresponding Merkle root is uniquely recoverable via inverse lookup in the pre-exposure ROM table. Consequently, the committed evaluation vector is causally frozen in the past prefix before the challenge digest is sampled.
3) **Telescopic Union of Algebraic and Field-Transport Exceptions (`CotaAdaptativaFinal`):** Across all $k \in \{4, \ldots, 12\}$, both FRI phases, and the AIR constraint combination, the union of exceptional challenge values in $\mathbb{F}_{p^2}$ prior to each E-draw has cardinality at most 130,709 (specifically 130,708 from the telescopic FRI sum plus 1 from the critical AIR cancellation set). Accounting for the modular reduction bias when mapping 256-bit digests into $\mathbb{F}_p^2$, each fresh challenge query incurs an exception rate bounded by

$$\rho = \frac{130{,}709}{p^2} + \frac{2{,}524{,}360}{10^{35}}, \tag{6}$$

yielding a global round-exception probability bounded by ρB across the entire history.

B. Coupled Bounded Selector ($R = 512$) and Grinding Composition

When the committed oracle is outside the proximity threshold (meaning fewer than $\lfloor 7 \cdot 2^{k+2}/8 \rfloor$ of the $M = 2^{k+2}$ pairs are "good" simultaneously for AIR and both FRI phases), the verifier accepts only if the 40 distinct pair indices extracted by the selector all land inside the good subset $G_T \cap G_C$, or if the selector distribution deviates from ideal sampling without replacement.

In `SelectorAcoplado` and `SelectorAdaptativo`, we couple the stateful byte-level selector (which squeezes up to $R = 512$ digests from the ROM to gather 40 distinct indices in $[0, 2^{k+2})$) with an idealized independent tape. Following Section 6.7.9 of the NEXUS thesis, for the Goldilocks prime $p = 2^{64} - 2^{32} + 1$ and shared budget $B = Q + 24{,}895$, in addition to ρ in (6), the stateful coupling error is

$$\delta(B) = \min\left\{1, \frac{512(B + 515)}{2^{256}}\right\}, \tag{7}$$

the exact hypergeometric omission probability of the 40-pair sample is

$$h_k = \frac{\binom{\lfloor 7 \cdot 2^{k+2}/8 \rfloor}{40}}{\binom{2^{k+2}}{40}}, \tag{8}$$

and the combined selector risk is

$$\sigma_k(B) = h_k + 2^{-300} + \delta(B). \tag{9}$$

In (9), h_k from (8) bounds the sample omission probability, 2^{-300} bounds the tail probability of failing to collect 40 distinct indices within 512 draws, and $\delta(B)$ from (7) bounds the stateful ROM coupling error over $B + 2$ effective steps.

```
@[expose] noncomputable def selector (k B : ℕ) : ℚ :=
  (Nat.choose (CardinalVista.umbral (k + 2)) 40 : ℚ) /
    Nat.choose (2 ^ (k + 2)) 40 +
    1 / (2 : ℚ) ^ 300 +
    AcoplamientoCadena.error Digest 512 (B + 2)

@[expose] noncomputable def cota512 (L bits k Q : ℕ) : ℚ :=
  let B := Q + 24895
  min 1 (((B * (49 * B + L) : ℕ) : ℚ) / (2 : ℚ) ^ 256) +
    CotaAdaptativaFinal.tasa * B +
    (1 / (2 : ℚ) ^ bits) * selector k B * (Q + 1)

theorem cota_512 {A : Type} (L bits k Q : ℕ)
    (proponer : Programa (List Octeto × A))
    (atacar : A → GramaticaTG2.Peticion → Programa A)
    (actualizar : A → GramaticaTG2.Peticion → Digest → A)
    (defecto : Digest)
    (hbits : bits = 12 ∨ bits = 20)
    (hk0 : 4 ≤ k) (hk1 : k ≤ 12)
    (hQ : ∀ H, Adversario.adversarias
      (oraculo (Adversario.ejecutar bits 512
        proponer atacar actualizar) defecto H)
      bits 512 proponer atacar actualizar ≤ Q) :
    let p := Adversario.ejecutar bits 512
      proponer atacar actualizar
    medida (fun H : universo p → Digest =>
      PoliticasFalsas.AceptaFalsa
        (oraculo p defecto H) bits 512 k
        proponer atacar actualizar) ≤
      min 1 (cota512 L bits k Q)
```

Listing 1. Main mechanized soundness bound in Lean 4 (`CotaTrabajoFinal.lean`).

Lemma 1 (Selector Risk per First Exposure)**.** *Conditioned on any fixed past prefix of at most B ROM entries and a fresh initial state/nonce pair, the probability that the $R = 512$ bounded selector extracts 40 distinct indices entirely within a subset of size at most $\lfloor 7 \cdot 2^{k+2}/8 \rfloor$ is bounded by $\sigma_k(B)$ as defined in (9).*

Crucially, in `CotaInicios` and `TrabajoSelectorLocal`, we prove two structural properties that prevent an adversary from amplifying this risk across verifier steps:

1) $Q+1$ **Eligible State/Nonce Pairs:** Every selector invocation is keyed by a distinct state/nonce absorption pair. Across Q adversarial queries and the final verifier execution (including any rejected attempts, duplicate queries, or interleavings), at most $Q+1$ distinct state/nonce pairs can ever be initiated.
2) **Local Grinding Composition (2^{-g}):** For $g \in \{12, 20\}$, the work check tests a fresh digest projection of mass 2^{-g} prior to index extraction. By proving the joint conditional law of two distinct fresh responses in the same finite ROM table, the local risk per eligible pair is bounded by $2^{-g}\sigma_k(B)$ without conditioning on future safe events.

C. Main Mechanized Soundness Theorems

Combining (5)–(9) yields our primary theorems in `Nexus.Verificador.CotaTrabajoFinal`, stated below and accompanied by their exact Lean 4 mechanization in Listing 1.

Theorem 1 (Fixed-Profile Byte-Level Soundness)**.** *For any adaptive adversary making at most Q oracle queries, any profile $k \in \{4, \ldots, 12\}$, grinding parameter $g \in \{12, 20\}$, and auxiliary bound L (with $B = Q+24{,}895$), the probability over the finite classical ROM that the byte-level verifier accepts a certificate for a false statement in profile k satisfies*

$$\Pr[\mathsf{AcceptFalse}_k] \le \min\Bigg\{1,\ \min\left\{1, \frac{B(49B+L)}{2^{256}}\right\} + \rho B + (Q+1)2^{-g}\sigma_k(B)\Bigg\}. \tag{10}$$

Theorem 2 (Adaptive Multi-Profile Soundness)**.** *If the adversary adaptively chooses which profile $k \in \{4, \ldots, 12\}$ to output at the end of its interaction, the false acceptance probability across all nine profiles satisfies*

$$\Pr[\mathsf{AcceptFalse}] \le \min\Bigg\{1,\ \min\left\{1, \frac{B(49B+L)}{2^{256}}\right\} + \rho B + (Q+1)2^{-g}\sum_{j=4}^{12}\sigma_j(B)\Bigg\}. \tag{11}$$

In (11), the hash-log exception and the ρB algebraic exception are charged once *globally across the shared ROM table rather than multiplied by nine.*

In `CotaTrabajoNumerica.lean`, we instantiate (10) and (11) using exact rational arithmetic (`norm_num`) without floating-point approximations:

- For $L = 10^6$, $k = 12$ ($T = 4096$ rows), $g = 20$ bits, and $Q = 2^{20}$ queries (`aceptacion_4096_release`), (10) yields
$$\Pr[\mathsf{AcceptFalse}_{12}] < \frac{4{,}757{,}335{,}454}{10^{12}} = 0.004757335454. \tag{12}$$
- Across all nine profiles adaptively, for $L = 10^6$, $g = 20$ bits, and $Q = 10^6$ (`aceptacion_release`), using the uniform upper bound $h_k \le 1/200$ in (11) gives
$$\Pr[\mathsf{AcceptFalse}] < \frac{1}{20} = 0.05. \tag{13}$$
- For the development profile $g = 12$ bits and $Q = 10^3$ (`ejemplo_debug`), (11) gives
$$\Pr[\mathsf{AcceptFalse}] < \frac{1}{80} = 0.0125. \tag{14}$$

IV. Bridging the Gap to Rust: Co-Design and Hardening

A central lesson of this work is that mechanizing the verifier in Lean 4 exposed concrete discrepancies in our initial Rust prototype (`nexus-zk`) that standard unit testing had missed.

A. Refuting the Legacy Verifier via Distance-7/8 Counterexamples

Prior to formalization, the legacy experimental STARK verifier in `nexus-zk` validated an isolated polynomial commitment without linking the composition polynomial in a dual-FRI pass and permitted flexible sample sizing.

TABLE II
CORRESPONDENCE BETWEEN LEAN 4 FORMAL MODULES AND HARDENED RUST IMPLEMENTATION

Verification Layer	Lean 4 Modules	Rust Modules	Enforced Security Invariant
1. Canonical Byte Parser & Pre-Hash Checks	`Nexus.Verificador.{Certificado, Forma, Completo}`	`nexus-zk::stark::{linked_codec, linked}`	Explicit little-endian decoding, canonical field check ($x < p$), strict EOF, and fixed profile-derived capacities before vector allocation (mitigating OOM DoS).
2. Fiat–Shamir Channel & Grinding Profile	`Nexus.Canal.{GramaticaTG2, ProgramaTG2, DesafiosPrograma}`	`nexus-zk::stark::{linked, channel}`	Domain separation tags, stage ordering, and strict enforcement of external caller grinding profile ($g \in \{12, 20\}$); certificates cannot downgrade g.
3. Bounded Ordered Selector ($R = 512$)	`Nexus.Canal.SelectorOrdenado`	`nexus-zk::stark::linked`	Extracts exactly 40 distinct pair indices in order of first appearance within $\leq$ 512 draws (counting duplicates); rejects immediately if draw 513 is needed.
4. Linked AIR & Dual-FRI Verification	`Nexus.Verificador.Consulta`, `Nexus.FRI.{ProgramaVerificacion, Recorrido, Cosets}`	`nexus-zk::stark::{linked, fri, merkle, field}`	Authenticates 6 trace positions against shared root, checks composition polynomial at x and $-x$, and folds both trace (k) and composition ($k+1$) over $\mathbb{F}_{p^2}$.
5. Application Context & State Registration	*Application wrapper outside core AIR theorem*	`nexus-active::{linked_verifier, verifier}`	`NitroVerifier::verify_expected` validates batch metadata against local snapshots prior to registration; prevents root overwrites and isolates optimistic mode.

During our S2 proximity transfer audit, we constructed three explicit adversarial traces whose evaluations were at relative Hamming distance 7/8 from the Reed–Solomon code of degree $< T$ (for $T = 16$ rows) yet corresponded to valid public statements and were *accepted* by the legacy verifier. To eliminate this class of vulnerabilities, we replaced the canonical verification path with the mechanized linked contract (`nexus-zk::stark::linked`).

B. Structural Alignment Between Lean 4 and Rust

Table II details the five enforcement layers synchronized between the Lean 4 specification and the hardened Rust crates (`nexus-zk` and `nexus-active`).

C. Hardening the Application Layer (`ActiveChain`)

Even a sound byte-level proof verifier can be subverted if the surrounding application layer accepts self-described statement headers from untrusted callers or mutates state before verification completes. In `nexus-active`, we connected `ActiveChain` to `NitroVerifier::verify_expected` under three invariants:

- **Private Snapshot Context:** `ActiveChain` retains private snapshots of locally constructed batches; the row profile and grinding parameter are fixed by node configuration rather than read from the untrusted proof envelope.
- **Pre-Registration Verification and Concurrency Control:** Validity and hybrid submission paths invoke `verify_expected` *before* recording any L1 submission or state root, serialized via a mutex that prevents duplicate batch-index registration under concurrent calls.
- **No Silent Fallbacks:** Calling validity or hybrid verification without an explicit expected context unconditionally rejects; legacy heuristic size checks and optimistic mode are strictly isolated from verified state roots.

In our consolidated workspace verification (executed offline with locked dependencies), all **223 global workspace tests** (and **436 tests** in the extended BFT journal suite) pass with zero failures, including 14 historical regression certificates (8 honest accepted, 6 far-from-code rejected) and 26 dedicated contract and integration test cases.

V. DISCUSSION: EPISTEMIC BOUNDARIES AND LIMITATIONS

A core methodological principle of our work is stating explicitly what our mechanized theorems *do* and *do not* establish:

1) **Finite Classical ROM vs. Concrete SHA3-256:** The bounds in (10) and (11) hold in the finite classical ROM where digests are sampled uniformly from $\{0,1\}^{256}$ with memory. The Rust implementation instantiates the channel with concrete `SHA3-256`. Standard ROM-to-concrete caveats apply: no mechanized proof can rule out structural algebraic exploitation of SHA3-256 without unprovable cryptographic assumptions.
2) **Parameter Regime vs. 100-Bit Security:** Equation (12) reflects a 40-query-pair configuration with $g = 20$ grinding bits and $Q = 2^{20}$ adversarial calls; it is an exact ceiling for that resource budget, *not* a claim of 2^{-100} cryptographic security. Achieving 2^{-100} soundness requires increasing the query count q, blowup factor, and grinding parameter g accordingly.

3) **Scalar AIR Recurrence vs. Full L2 Transaction Semantics:** The mechanized AIR encodes a scalar algebraic recurrence over $\mathbb{F}_p$. It does *not* arithmetize full SHA3-256 state roots or ML-DSA-65 signature verification inside the circuit. Consequently, in `ActiveChain`, real transaction batches carrying out-of-model SHA3 roots are cleanly rejected by the scalar contract rather than truncating hashes or fabricating synthetic proofs.
4) **Absence of Universal Machine-Code Refinement:** While we statically audited the line-by-line correspondence between the 88 Rust source files and 354 Lean files (verifying absence of $u128$ overflow since $(p-1)^2 < 2^{128}$ and $2(p-1) < 2^{128}$), we do not use a verified compiler or a formal Rust-to-Lean extractor (such as Aeneas or `coq-of-rust`).
5) **Quantum Random Oracle Model (QROM):** Extending the adaptive multi-round FRI/AIR reduction to superposition queries in the QROM (accounting for compressed-oracle recording losses [8], [14]) remains open future work.

VI. Related Work and Conclusion

Recent efforts such as **ArkLib** [2] in Lean 4 and formalizations of zero-knowledge protocols in Coq, EasyCrypt, and Isabelle/HOL [1], [3], [9], [11], [13], [15] focus primarily on modular algebraic reductions for IOPs, polynomial commitment schemes, or low-level field arithmetic. Our work complements these efforts by mechanizing the *concrete byte-level Fiat–Shamir verifier* end-to-end (2,709 axiom-clean declarations across 347 submodules), including deterministic binary deserialization, stateful ROM lookup tables with collision/cycle detection (SinBad in (5)), bounded deduplication loops ($R = 512$) in (9), and explicit rational arithmetic bounds in (10)–(14) via `norm_num`.

By coupling the byte parser, stateful ROM history, and g-bit grinding over $Q + 1$ eligible pairs, our formalization eliminates idealized independence assumptions and guided the concrete hardening of our Rust verifier in the NEXUS framework. Future work will address QROM superposition bounds for linked FRI and mechanized translation validation between Rust and Lean 4.

References

[1] J. B. Almeida, M. Barbosa, G. Barthe, A. Blot, B. Grégoire, V. Laporte, T. Oliveira, H. Pacheco, B. Schmidt, and P.-Y. Strub, "Jasmin: High-assurance and high-speed cryptography," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2017, pp. 1807–1823.

[2] ArkLib Contributors, "ArkLib: A formal library of interactive oracle proofs and arguments of knowledge in Lean 4," GitHub repository, 2024.

[3] G. Barthe, B. Grégoire, S. Heraud, and S. Zanella-Béguelin, "Computer-aided security proofs for the working cryptographer," in *Advances in Cryptology – CRYPTO 2011*, Springer, 2011, pp. 71–90.

[4] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev, "Fast Reed–Solomon interactive oracle proofs of proximity," in *Proc. 45th Int. Colloq. Automata, Languages, and Programming (ICALP)*, 2018, pp. 14:1–14:17.

[5] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev, "Scalable, transparent, and post-quantum secure computational integrity," *IACR Cryptology ePrint Archive*, Rep. 2018/046, 2018.

[6] E. Ben-Sasson, D. Carmon, Y. Ishai, S. Kopparty, and S. Saraf, "Proximity gaps for Reed–Solomon codes," in *Proc. IEEE 61st Annu. Symp. Found. Comput. Sci. (FOCS)*, 2020, pp. 900–909.

[7] E. Ben-Sasson, A. Chiesa, and N. Spooner, "Interactive oracle proofs," in *Theory of Cryptography Conf. (TCC 2016-B)*, Springer, 2016, pp. 31–60.

[8] A. Chiesa, P. Manohar, and N. Spooner, "Succinct arguments in the quantum random oracle model," in *Theory of Cryptography Conf. (TCC)*, Springer, 2019, pp. 1–29.

[9] A. Erbsen, J. Philipoom, J. Gross, R. Sloan, and A. Chlipala, "Simple high-level code for cryptographic arithmetic – with proofs, without compromises," in *Proc. IEEE Symp. Security and Privacy (SP)*, 2019, pp. 1202–1219.

[10] A. Fiat and A. Shamir, "How to prove yourself: Practical solutions to identification and signature problems," in *Advances in Cryptology – CRYPTO '86*, Springer, 1986, pp. 186–194.

[11] L. Goldberg, S. Papini, and M. Riabzev, "Cairo – a Turing-complete STARK-friendly CPU architecture," *IACR Cryptology ePrint Archive*, Rep. 2021/1063, 2021.

[12] U. Haböck, "A summary on the FRI low degree test," *IACR Cryptology ePrint Archive*, Rep. 2022/1216, 2022.

[13] A. Lochbihler, S. R. Sefidgar, D. Basin, and U. Maurer, "Formalizing constructive cryptography using CryptHOL," in *Proc. IEEE 32nd Comput. Security Found. Symp. (CSF)*, 2019, pp. 152–166.

[14] M. Zhandry, "How to record quantum queries, and applications to quantum indifferentiability," in *Advances in Cryptology – CRYPTO 2019*, Springer, 2019, pp. 239–268.

[15] J.-K. Zinzindohoué, K. Bhargavan, J. Protzenko, and B. Beurdouche, "HACL*: A verified modern cryptographic library," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2017, pp. 1789–1806.